SLIDE TYPE GALLERY

# 사내 문서에 답하는 에이전트

발표에서 자주 나오는 장 22가지를 한 주제로 채운 견본

> 장마다 패턴 하나. 같은 이야기를 이어 붙였다.

견본 패턴 A · 표지

AGENDA

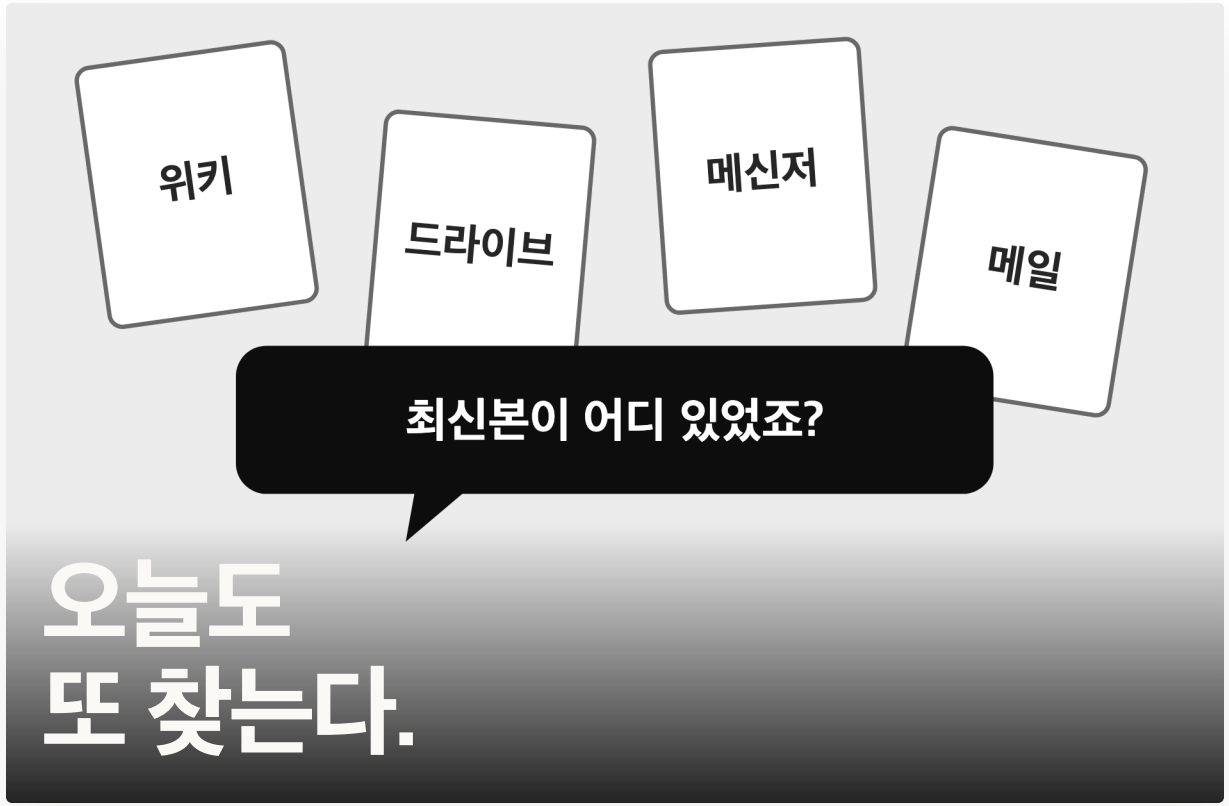
# 오늘은 네 덩어리로 이야기한다

| | | |
|---|---|---|
| 01 | **왜 필요한가**<br>모델만 쓸 때의 약점 | 1분 |
| 02 | **어떻게 동작하나**<br>검색과 에이전트 구조 | 2분 |
| 03 | **무엇이 좋아졌나**<br>정답률과 질문 유형별 결과 | 2분 |
| 04 | **어떻게 도입하나**<br>일정과 결정 요청 | 1분 |

**견본** 패턴 U · 시작용 목차. 덩어리가 바뀌면 같은 목록에서 .now 를 옮겨 다시 띄운다

PROBLEM

# 문서는 있는데, 찾을 수가 없다




필요한 규정이 저장소 네 곳에 흩어져 있다.

> **“휴가 규정 최신본이 어디 있었죠?”**

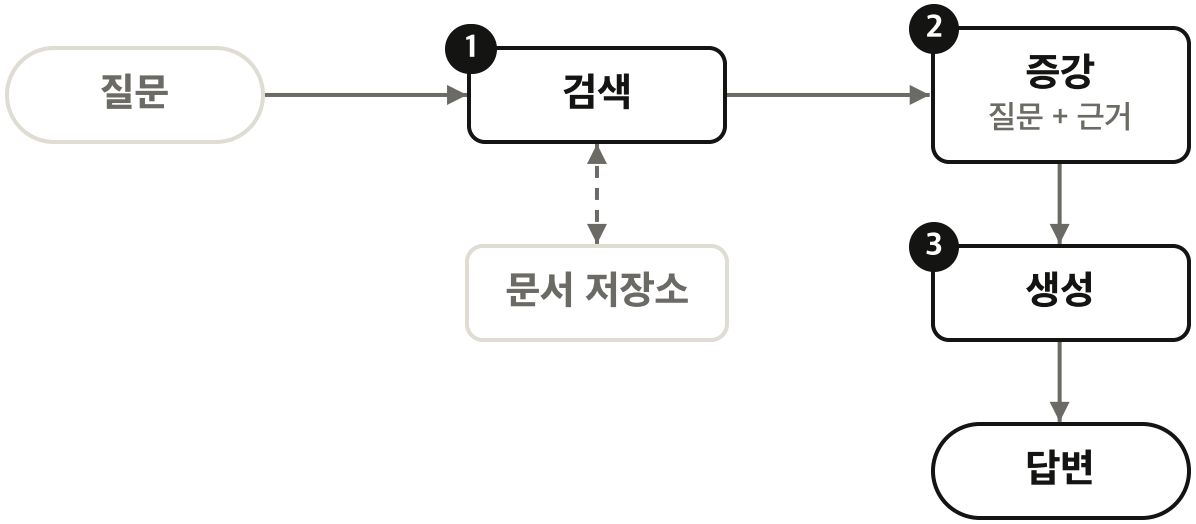
- 같은 질문이 매주 반복된다
- 개정 전 문서로 답한다
- 담당자 한 명에게 질문이 몰린다
- 찾다가 포기하고 추측한다

**찾는 시간이 일하는 시간을 먹는다.**

**견본** 패턴 C · 공감해야 하는 상황은 장면으로

CONCEPT

# RAG는 찾고, 붙이고, 답한다




**1 검색 · Retrieval**

질문과 뜻이 가장 가까운 문서 조각을 저장소에서 찾는다.

**2 증강 · Augmented**

찾은 조각을 원래 질문과 합쳐 프롬프트(모델에 보낼 입력)를 채운다.

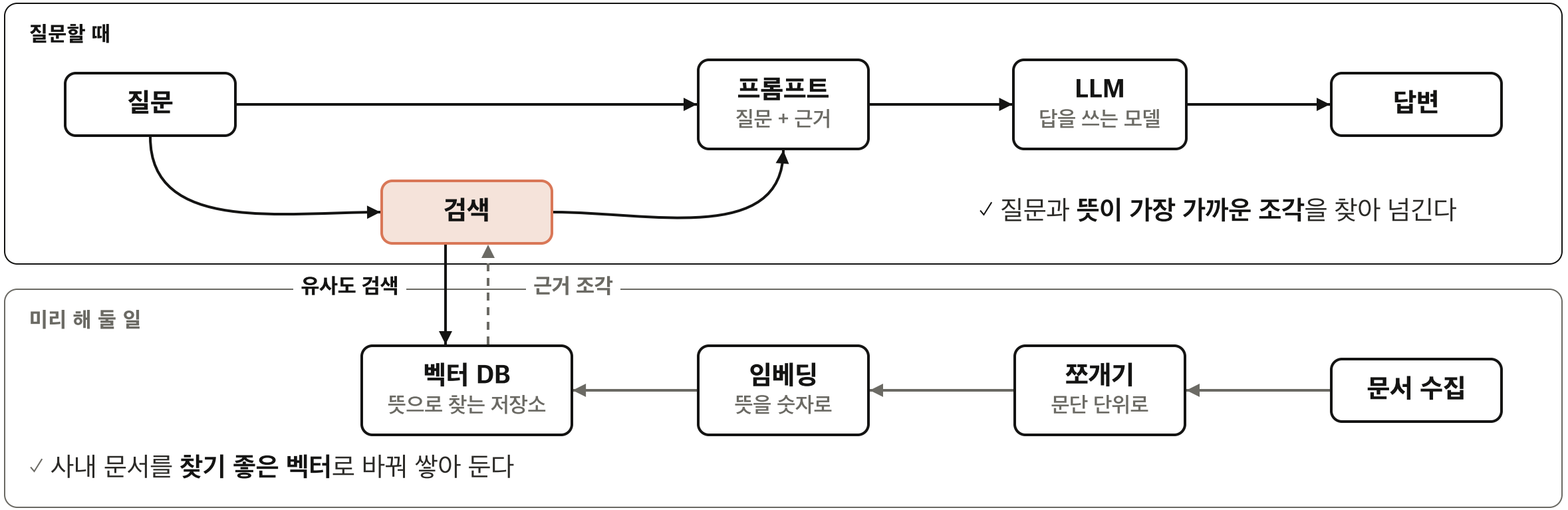
**3 생성 · Generation**

채워진 프롬프트로 모델이 근거 있는 답을 쓴다.

**견본** 패턴 Q · 번호 = 범례 = 말하는 순서

STRUCTURE

# 미리 쌓아 둔 문서에서 질문마다 근거를 꺼낸다




**견본** 패턴 O · 무거운 일은 미리, 질문 때는 찾기만

TWO MODES

# 새 문서는 바로, 고친 문서는 매일 밤 다시 색인한다

MODE A · 문서가 올라올 때

## 새 문서는 바로 검색 저장소에 넣는다

업로드 감지 → 쪼개기 → 임베딩 → 저장

올린 지 몇 분 안에 답에 쓰인다.

MODE B · 매일 밤

## 고친 문서만 골라 다시 넣는다

변경 비교 → 바뀐 문단만 → 다시 임베딩 → 교체

밤사이 교체가 끝나면 개정 전 문단은 답에서 빠진다.

**견본** 패턴 J · 갈라짐·왕복이 없는 직선 흐름이면 J로 충분하다

CODE · 2 / 5

# 검색 노드는 찾은 조각과 횟수만 돌려준다

1. State 정의
2. **검색 노드**
   - state: 노드들이 함께 쓰는 메모
   - 돌려준 칸만 바뀐다
3. 답변 노드
4. 그래프 연결
5. 실행

```python
# 검색 노드: 질문으로 문서 조각을 찾는다
def retrieve(state: DocState):
    question = state["messages"][-1].content
    chunks = vector_db.similarity_search(question, k=3)
    return {
        "context": chunks,                                  (1)
        "search_count": state["search_count"] + 1,          (2)
    }
```

**(1) 근거 조각 3개를 넘긴다**
`context` 비어 있음 → 조각 3개. 답변 노드가 읽는다.

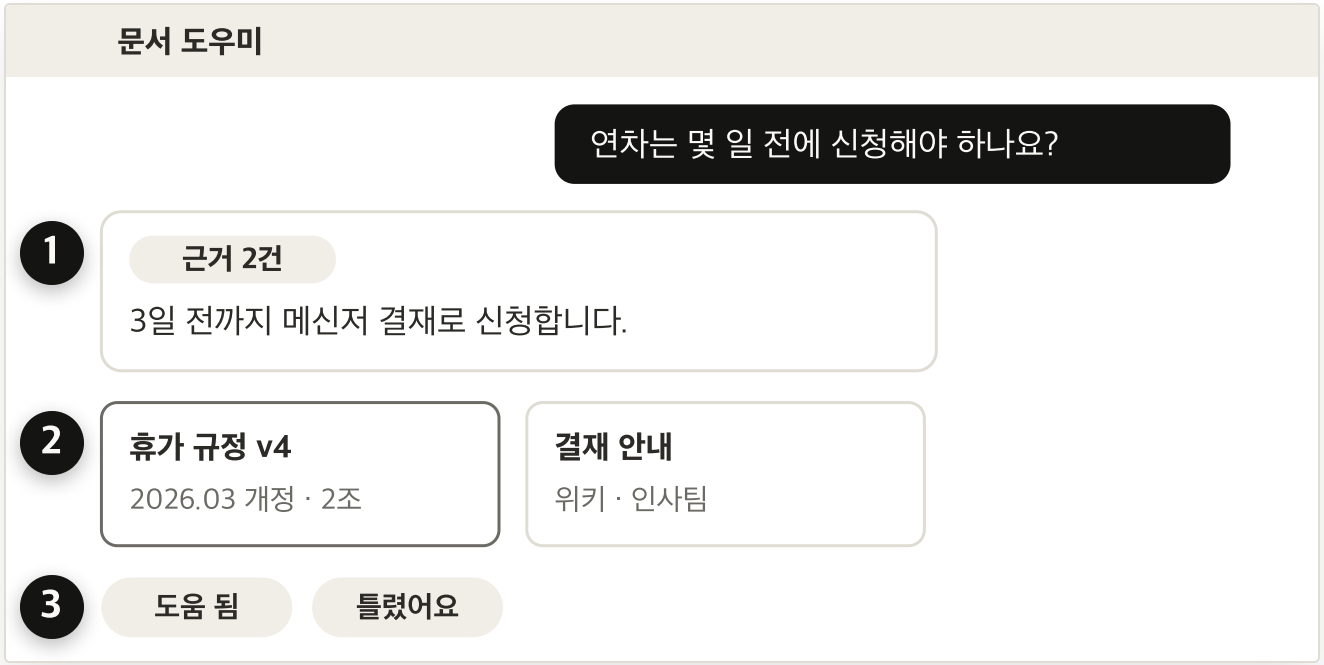
**(2) 검색 횟수를 1 올린다**
`search_count` 2 → 3

**견본** 패턴 S · 레일은 지금 단계만, 강조 줄 번호 = 결과 번호

PRODUCT

제안

# 답과 함께 근거 문서를 열어 볼 수 있다




예시 화면 · 실제 제품 캡처로 교체한다.

**1 답변과 근거 수**
답 위에 근거를 몇 건 찾았는지 먼저 보인다.

**2 근거 문서 카드**
개정일과 조항이 보이고, 누르면 그 조항의 원문이 열린다.

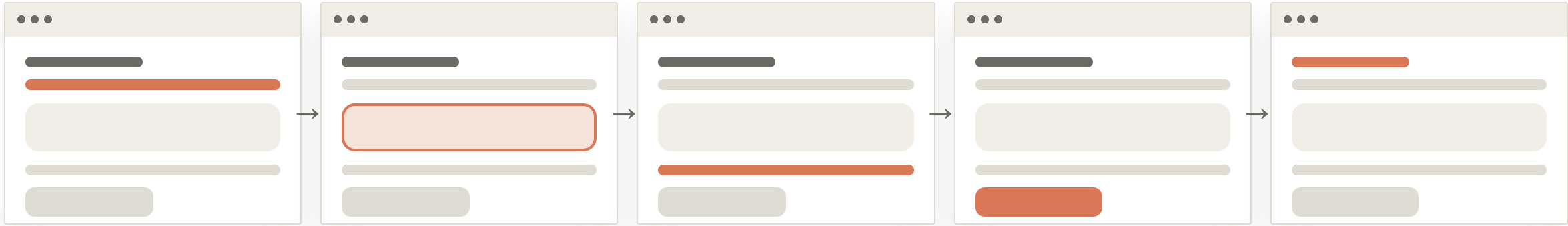
**3 피드백**
틀린 답은 한 번에 신고하고, 문서 담당자에게 간다.

**견본** 패턴 M · 관심 영역만 잘라 핀으로 짚는다

JOURNEY

제안

# 질문부터 문서 수정까지 한 흐름으로 이어진다



**1 질문**
메신저에서 바로 묻는다

**2 답변**
근거 2건과 함께 받는다

**3 원문**
해당 조항이 열린다

**4 신고**
틀린 답을 누른다

**5 수정**
담당자가 문서를 고친다

**견본** 패턴 L · 단계마다 바뀌는 곳 하나만 칠한다. 화면이 없으면 와이어프레임으로

CONTRACT

제안

# 사용자 행동 넷이 API 넷으로 이어진다

| | | | |
|---|---|---|---|
| 1 **질문 보내기** 메신저 · 웹 | 1 | POST | /ask | 답변과 출처 목록 |
| 2 **근거 열기** 출처 카드 클릭 | 2 | GET | /docs/:id | 근거 문서 원문 |
| 3 **답변 평가** 도움 됨 · 틀렸어요 | 3 | POST | /feedback | 답변 평가 저장 |
| 4 **문서 갱신** 담당자가 수정 | 4 | POST | /reindex | 바뀐 문서 다시 색인 |

**견본** 패턴 K · 행동과 API 행에 같은 번호. 표는 한 행만 짚는다

CONTRAST

# 찾는 일은 줄고, 판단하는 일은 남는다

↓ 줄어드는 일

## 찾기

- 위키·드라이브를 뒤지는 시간
- 담당자에게 묻는 메신저
- 같은 질문에 반복해서 답하기

에이전트가 대신 찾는다.

↑ 남는 일

## 판단

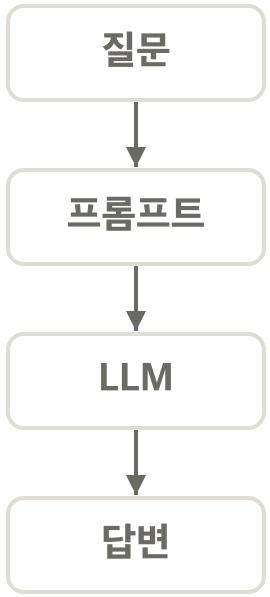
- 근거가 맞는지 확인하기
- 규정이 바뀌면 문서 고치기
- 문서에 없는 예외 결정하기

**사람은 판단에 시간을 쓴다.**

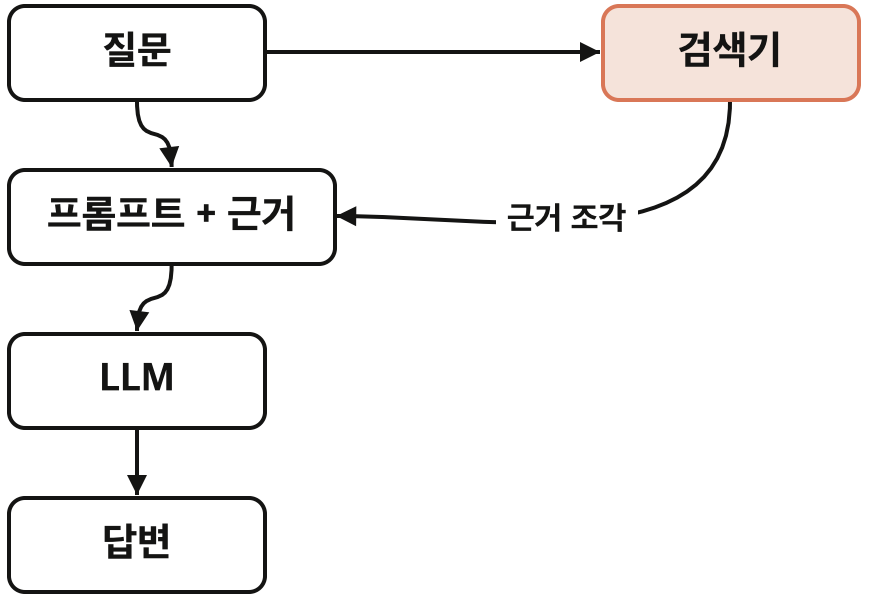
BEFORE → AFTER

# 검색기를 더해 프롬프트에 근거를 넣는다

■ 모델만 쓸 때




■ 문서 검색을 붙이면




**모델은 그대로 두고, 질문과 함께 찾은 문서 조각을 넘긴다.**

**견본** 패턴 P · 나머지 흐름은 바뀌지 않는다

WHY

# 문서 검색을 붙이면 여섯 약점이 한 줄씩 줄어든다

| “모델 지식에만 기대면” | “문서 검색을 붙이면” |
| --- | --- |
| **지식 시점** : 학습 이후 정보는 모른다 | **최신 정보** : 문서만 바꾸면 반영된다 |
| **환각** : 없는 사실을 그럴듯하게 만든다 | **근거 답변** : 찾은 문서를 근거로 답한다 |
| **도메인 부족** : 회사 사정을 모른다 | **사내 특화** : 내부 문서를 그대로 쓴다 |
| **출처 불명** : 어디서 온 말인지 모른다 | **출처 표시** : 근거 문서를 함께 보여준다 |
| **갱신 부담** : 모델 지식을 바꾸려면 재학습한다 | **쉬운 갱신** : 문서는 다시 색인하면 된다 |
| **비용** : 재학습 비용이 크다 | **비용 절감** : 재학습 없이 붙인다 |

**견본** 패턴 R · 약점마다 짝이 되는 해결이 하나씩 있다

PROBLEM → FEATURE

# 문제 다섯에 기능 다섯을 하나씩 댄다

지금 겪는 문제

| P1 | P2 | P3 | P4 | P5 |
|---|---|---|---|---|
| **흩어짐** | **낡은 답** | **출처 없음** | **쏠림** | **방치** |
| 저장소 네 곳 | 개정 전 문서 인용 | 근거를 못 댄다 | 담당자 한 명 | 틀려도 못 고친다 |
| ↓ | ↓ | ↓ | ↓ | ↓ |
| 대응하는 기능 | | | | |
| 통합 색인 | 최신본 우선 검색 | **출처 카드** | 메신저 봇 | 피드백 신고 |

**→ 출처 카드가 근거를 확인하는 시간을 줄인다.**

**견본** 패턴 D · 칸마다 라벨 수준만, 문장은 R로

KEY NUMBER

# 정답률이 54%에서 81%로 올랐다

**81%**

**사내 규정 질문 정답률**

같은 질문 200건을 두 방식으로 묻고 사람이 채점했다.

| | |
|---|---|
| 모델만 | 54% |
| **문서 검색을 붙이면** | **81%** |

예시 데이터 · 레이아웃 견본용 수치

**견본** 패턴 V · 숫자는 비교 기준과 함께

TABLE

# 벡터 검색이 정확도와 속도의 균형점이다

| 방식 | 정답률 | 출처 제시 | 응답 시간 | 비고 |
|---|---|---|---|---|
| **모델만** | 54% | 0% | 1.2초 | 근거 없음 |
| **키워드 검색 + 모델** | 68% | 92% | 1.8초 | 동의어에 약함 |
| **벡터 검색 + 모델** | **81%** | **97%** | **2.1초** | **이번 제안** |
| **벡터 + 재순위** | 84% | 97% | 3.4초 | 찾은 결과를 다시 정렬 · 느림 |

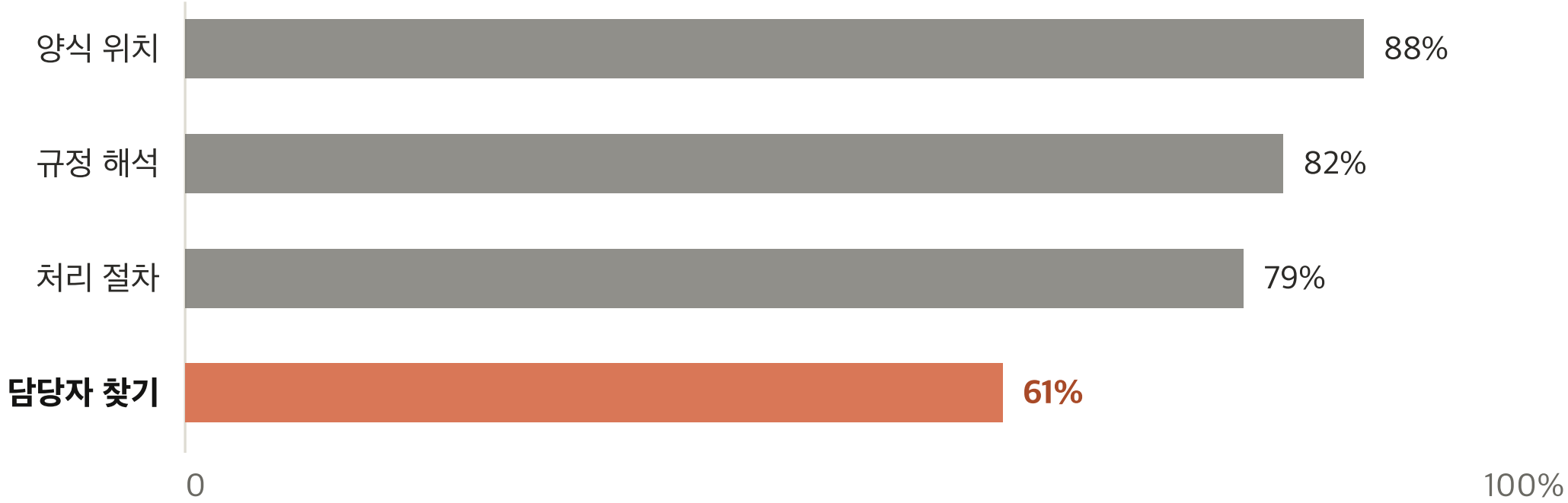
**벡터 검색은 재순위보다 3%p 낮지만 1.3초 빠르다.**

예시 데이터 · 레이아웃 견본용 수치

ANALYSIS

# 담당자를 찾는 질문만 정답률이 낮다

질문 유형별 정답률 · 예시 데이터




**원인 가설: 담당자 정보가 문서에 없다. 조직도를 색인에 더해 확인한다.**

출처: 예시 데이터 · 레이아웃 견본용 수치

**견본** 패턴 N · 그림을 자르지 않고 온전히. 인라인 SVG면 테마 색을 따른다

COMPONENTS

# 다섯 부품 중 둘은 동작하고 셋은 만든다

수집 → 색인 → 검색 → 질문 창구 → 피드백

collector
**수집기**
위키 연결
● 현재 확인됨

indexer
**색인기**
쪼개기·임베딩
● 현재 확인됨

retriever
**검색기**
벡터 검색 + 최신본 우선
● 제안

bot
**메신저 봇**
질문 창구
● 제안

dashboard
**피드백 보드**
틀린 답 모아 보기
○ 예정

**견본** 패턴 H · 범례와 카드 색을 맞춘다

● 현재 확인됨 ● 제안 ○ 예정

WHO FIRST

# 가장 먼저 바뀌는 사람은 신규 입사자다

★

## 신규 입사자

규정을 옆자리에 묻는다

**첫 주 질문을 봇이 받는다**

**질문이 가장 많고, 문서 위치를 모른다.**

인

### 인사 담당

같은 질문에 반복 답변

**예외만 답한다**

팀

### 팀장

규정 확인을 대신 해 줌

**링크 하나로 끝**

문

### 문서 관리자

낡은 문서를 모른다

**신고가 모인다**

보

### 보안 담당

문서 유출 걱정

**권한별 검색**

**견본** 패턴 E · 균등하게 나열하지 않는다

ROLES

# 에이전트는 찾고, 사람은 결정한다

에이전트

## 찾고 초안 쓰기

- 문서 검색과 요약
- 근거를 붙인 답변
- 모르면 모른다고 답하기

답마다 출처를 단다.

+

사람

## 확인하고 결정하기

- 근거가 맞는지 확인
- 규정 예외 판단
- 문서 수정 승인

**규정 해석은 사람이 한다.**

**견본** 패턴 F · 역할 라벨은 상태 배지와 다른 토큰

ROADMAP

# 위키에서 시작해 한 분기 안에 전사로 넓힌다

1 — 2 — 3

1단계 · 완료

## 위키 색인과 채점

- 위키 3,000쪽 색인
- 질문 200건 채점

2단계 · 승인 대기

## 메신저 봇 시범 4주

- 인사팀 한 곳에서
- 피드백 수집

3단계 · 분기 말

## 전사 확대

- 드라이브 연결
- 조직도 색인

**견본** 패턴 G · 지금 단계 하나만 칠한다

DECISION

# 4주 시범을 승인하고, 이 네 지표로 판단하자

요청하는 결정

**1. 메신저 봇 시범 4주 · 인사팀 한 곳 · 종료 시 네 지표로 판정**

**2. 인사팀 문서 읽기 권한**

**3. 채점 인력 주 2시간**

정확도

**정답률 85% 이상**

매주 질문 50건을 채점한다.

부담

**재문의 30% 감소**

도입 전 4주 대비, 담당자에게 다시 물은 비율.

근거 활용

**근거 확인 40% 이상**

답변 중 근거 문서를 연 비율.

운영

**반영 하루 이내**

문서를 고친 뒤 답에 쓰이기까지.